

CANDY DIALOGUE ENGINE

Getting Started

1.1.0

This document will give you an overview of Candy Dialogue Engine, so you have a general idea of how it works before you read the more detailed documentation on its features.

Before proceeding, you need to install Candy Dialogue Engine in your project. If you haven't done so, read **Installation_Instructions.pdf** first.

Two Parts

The first thing to understand is that Candy Dialogue Engine comes in two separate parts: the code that runs dialogues in your game, and the software that you use to create dialogues.

- **Candy Dialogue Engine (DE)** is the code that runs dialogues in your project. It's made of scripts and scenes, and you add it inside your Godot project.
- **Candy Dialogue Creator (DC)** is a standalone software, made in Godot, which provides a UI and useful features for creating and editing dialogues. It can read your project files to list and display data, such as the characters (actors) in your game, image files for portraits, etc. When you've written dialogues in Candy DC, you can export them directly to your project.

This split architecture has multiple benefits over a GUI built into Candy Dialogue Engine and displayed inside the Godot Editor: it's more stable and works more reliably, it's not constrained inside the Godot Editor's UI, and it allows users to edit dialogues without opening Godot or having access to the project files.

You can download Candy Dialogue Creator as a compiled executable for Linux, Windows and MacOS, or as a raw (uncompiled) Godot project on [GitHub](#).

Five Components

There are five components to Candy Dialogue Engine that you need to understand

First, there's the **dialogues**: they're created and edited in the Candy Dialogue Creator, and exported as text (.txt) files.

Dialogues are like a script: they are made up of dialogue lines, which act as instructions for the dialogue engine. The engine reads the lines in order and does what they tell it to do: display speech, play a media, pause dialogue, modify variables, jump to another dialogue line, etc.

In simple terms: everything Candy Dialogue Engine can do is done through dialogues.

To create a dialogue, you just add the lines you need, then fill in data to configure them.

Second, there's the **Candy_DE folder**: it's where the scripts, scenes and resources connected to Candy Dialogue Engine live.

If you create a UI for dialogues, or if you have media files that dialogues should display (such as portraits, cutscene videos, voices, background audio, etc.) you would place them in the Candy_DE subfolders. Dialogues are also placed in a subfolder of Candy_DE.

Candy Dialogue Engine offers the option to use your own custom folder structure if you prefer, but this is not recommended as it can require some manual work when updating.

Third, there's the **User Data scripts**. These scripts contain variables or functions that are designed to be customized by users, and which the dialogue engine uses. There are four such scripts:

- **Candy_Properties.gd** contains the core engine settings, such as the language the game is set to, the paths to various media resources like portraits or voice files, and more.
- **Candy_Database.gd** contains data about the game itself. As an example, it features a dictionary you can fill with data about the characters in your game: how to display their name, whether do use a special color when they speak, etc.
- **Candy_Functions.gd** contains *hook* functions: the engine calls these automatically at specific moments, but they don't do anything by default. Instead, users can write custom code inside these hook functions, to create any custom behavior they want.
- **Candy_Localizations.gd** contains localization tables: these are dictionaries that tell the engine how to translate things like character names or UI text for each language.

NOTE: These scripts extend one-another, and Candy_Functions.gd is an autoload (candy_de). This means your code can access any variable or function in all four of these scripts by using the **candy_de** prefix. For example: **candy_de.your_variable** or **candy_de.your_function()**

Fourth, there's the **dialogue UI**. The dialogue UI actually isn't a single scene: it's made of multiple UI elements. For example, one such element might be a dialogue box, and another might be a choice menu.

These UI elements can be placed anywhere in your project's node tree, as long as you tell the engine where to find each of them. By keeping them separate from each other, it makes it easier to customize them.

Last, there's the dialogue engine itself: this is the **Candy_Engine.gd** script, which processes dialogues.

It contains a function you can call to load a dialogue from a .txt file, and a second function you can call to start the loaded dialogue. Simple. You can even tell it at which line to start.

The engine will read through the loaded dialogue, one line at a time. It does whatever the current line is configured to make it do, then moves on to the next one.

Depending on the lines, it may call functions or modify variables inside UI elements. For example, to display spoken text, the dialogue engine reaches a RichTextLabel node inside the dialogue box and writes the text inside of it.

The engine will also call functions or read/modify variables in the User Data scripts mentioned previously. Sometimes the engine will look for resource files in the Candy_DE subfolder, and sometimes the UI elements themselves will do it.

As you can see, the engine script is the central part that ties everything together.

Candy UI

The installation instructions told you to add the Candy_UI scene as an autoload, and to use it as the general UI for dialogues. It's useful to understand what the Candy_UI really is:

- The Candy_UI scene is made of a **CanvasLayer** root node, which means it can display its contents in front of the camera.
- This root node has the **Candy_Engine.gd** script attached.
- The root node also has various **dialogue UI elements** as children.

It's basically an all-in-one scene that can run and display dialogues. Because it's an autoload, it's always present and you can call on it at anytime in your game.

Dialogue Engine Instances

The Candy_UI is just one instance of the dialogue engine.

Anytime you have a node with the Candy_Engine.gd script attached, this node can load and run its own dialogues independently.

This allows you to run multiple dialogues at the same time: if you had 10 such nodes, they could each run their own dialogue without conflicting with each other.

Nodes with the Candy_Engine.gd script attached are called **dialogue engine instances**.

To work properly, each engine instance should have its own personal UI elements: they don't need to be children of the engine instance (although this probably makes the most sense in most cases), but you need to provide the paths to these UI elements to each engine instance (this can be done in the Inspector, as the paths are exported variables).

Dialogue engine instances can be autoloads (like Candy_UI) but this is not required.

Is it useful to have multiple dialogue engine instances? That depends on your game:

- For most games, a single general instance, such as Candy_UI, can be enough.
- You may want to have 2 or 3 instances: for example, one instance for exploration mode, and one for combat mode - each with its own settings and UI style.
- For character 'barks' (background comments made by NPCs), you would assign each NPC its own engine instance and make your game use these whenever an NPC should produce a 'bark'.
- For modding, you would make your game load dialogue .txt files made by players, and run each such dialogue inside a separate engine instance.

It can feel a bit advanced if you're new to Godot, but don't worry: to get started, the Candy_UI will be enough.

Engine Configuration

There are two kinds of configuration parameters for the dialogue engine.

The first type of parameters are engine **properties**, found in the Candy_Properties.gd script. These are shared between all dialogue engine instances.

The second type are engine **settings**. These are found in the Candy_Engine.gd script itself, and are exported variables.

If you're a beginner in Godot: exported variables can be modified in the Inspector. The great thing about this is that if multiple nodes share the same script, the modifications only apply to the node you modify, not the others.

Thanks to this, each engine instance can be configured differently: it can have its own UI elements, its own dialogue mode, its own portrait settings, etc.

Of course, you can also write code to modify the settings of an instance at runtime (and dialogues can do so as well), so you don't need multiple engine instances if you just plan to change a few settings from time to time.

NOTE: to modify engine settings through dialogues, use the \$Set command. It can be useful to let each dialogue configure the engine at dialogue start, instead of writing code that configures the engine depending on which dialogue is loaded. In the future, we will likely add a dedicated command to more easily modify engine settings through dialogue.

Candy DE Symbols

VERY IMPORTANT

Dialogues may require that you use special symbols for some data. This includes the following:

- Various symbols to designate variable references inside line data.
- A symbol to designate character roles.
- Several symbols used in spoken text, for displaying variable values or for the word substitution feature.

Don't worry if this feels vague for now. The important thing is that before you start writing dialogues, you remember to check the default symbols and customize them to use characters you might find more convenient.

These symbols are configured in Candy_Properties.gd. See the **Variables.pdf** guide to learn what they do and how to modify them.

Modifying symbols after you've written dialogues will require that you change the symbols everywhere you used them, which can be tedious work.

Summary

Dialogue engine: Candy_DE/Scripts/Candy_Engine.gd

- ➔ Assign to nodes. Each node is a separate engine.
- ➔ Use for running dialogues.
- ➔ Customize settings in the Inspector.

Dialogues: Candy_DE/Dialogues

- ➔ Create in Candy Dialogue Creator.
- ➔ Export to your Godot project as .txt files.
- ➔ Load in the dialogue engine to run.

Resources: Candy_DE/Media

- ➔ Various subfolders for organizing files.
- ➔ Store portraits, busts, voice, sprites, images, videos, audio and backgrounds.
- ➔ Files are retrieved by the dialogue engine or the UI as needed.
- ➔ See the guides relevant to each feature for details.

UI elements: Candy_DE/Scenes

- ➔ Each UI element is a scene. Create as many as you want.
- ➔ Contain code for performing some of the work.
- ➔ Instantiate UI elements anywhere in your node tree.
- ➔ Assign each dialogue engine UI instances to use.

User data: Candy_DE/Scripts/User_Data

- ➔ Scripts (.gd files).
- ➔ Custom data about the project, hook functions, translation tables and engine properties.
- ➔ Called upon during dialogues by the engine/UI elements as needed.
- ➔ Referenced with autoload prefix 'candy_de'.

Symbols: Candy_Properties.gd

- ➔ Choose symbols convenient to type.
- ➔ Best done early in a project.
- ➔ See the 'Variables.pdf' guide for important guidelines.